Arquitectura do Sistema de Armazenamento em Nuvem

Documento de arquitectura — Requisitos de referência: RF01–RF30, RNF01–RNF10 (`requisitos_sistema_armazenamento_`)

1. Sumário executivo

O **Sistema de Armazenamento em Nuvem** que permite a um utilizador comum **ceder espaço de seu próprio computador** ao sistema e, em troca, usar o espaço disponibilizado por outros para guardar os seus dados — um modelo de armazenamento em nuvem em que cada participante é simultaneamente fornecedor e consumidor de espaço, com custo marginal próximo de zero.

2. O problema

2.1 As três dores

#	Dor	Evidência
P1	Custo recorrente	Modelo de negócio de todos os fornecedores mainstream
P2	Desperdício assimétrico	Qualquer portátil actual
P3	Dependência de fornecedor	Concentração do mercado em 3–4 operadores

2.2 A complicação técnica

Aqui está a complicação: "usar o disco do utilizador" — falha em três frentes, e é

1. **Confiança.** Quem cede 100 GB do seu PC não pode ler ficheiros de outras
1. **Disponibilidade.** Um PC pessoal desliga-se, dorme, muda de rede
- **Alcance de rede.** Um PC doméstico está quase sempre atrás de

arquitectura tem de ser pensada em segurança, consistência e disponibilidade *representam uma*

3. A solução

3.1 Conceito central: o utilizador duplo

Utilizador (secção 0.2) **papéis** papéis, não contas (ver histórias de

```
                +-----------------------------+
                |      Conta de utilizador    |
                +--------------+--------------+
                               |  acumula papéis
        +----------------------+----------------------+
        v                      v                      v
+---------------+    +-------------------+    +--------------+
| Cliente Comum |    |  Cliente Provedor |    | Administrador|
|  (consumidor) |    |  (cede espaço)    |    |  (governança)|
+---------------+    +-------------------+    +--------------+
```

3.2 Os dois espaços unificados

Tipo	Dono	Significado
`SYSTEM`	`null` (partilhado)	Nuvem oficial — sempre disponível
`DEVICE`	utilizador	PC pessoal do Cliente Provedor

```
Upload --> destino escolhido
           |
           +- AUTO   --> política padrão do utilizador (RF22)
           |             ex.: "preferir dispositivo pessoal"
           |
           +- SYSTEM --> nuvem oficial (sempre online)
           |
           +- DEVICE --> nó pessoal específico (tem de estar online)
```

3.3 Fluxo de vinculação de um dispositivo (RF17)

```
+--------------+                      +--------------+        +--------------+
|  Frontend    |                      |   Backend    |        |  Agente (PC) |
+------+-------+                      +------+-------+        +------+-------+
       |  POST /nodes/pairing-code            |                       |
       |------------------------------------->|  código de 15 min     |
       |<-------------------------------------|  (uso único)          |
       |                                      |                       |
       |  utilizador corre: bash install-node.sh A7K9M2P4              |
       |-------------------------------------------------------------->|
       |                                      |                       |
       |            POST /nodes/register      |                       |
       |            { name, localPath,        |                       |
       |              totalSpaceBytes,        |                       |
       |              pairingCode }           |                       |
       |<-------------------------------------+-----------------------+
       |                                      |                       |
       |                            consome código, cria StorageNode   |
       |                            (type=DEVICE, status=ONLINE)       |
       |                            cifra deviceToken                  |
       |                                      |                       |
       |<-- { nodeID, deviceToken } -----------+-----------------------+
       |                                      |                       |
       |                                      |   agente guarda token |
```

```
        |                                       |   e sobe servidor HTTP|
        |                                       |   local na porta 8081 |
```

Segurança após pareamento: ao utilizador. Segurança do pareamento: é de uso único e é consumido pelo código

3.4 Fluxo de escrita de um ficheiro (replicação)

```
ReplicationService.replicateFile(fileID, owner, containerData, dest, targetNodeId)
    |
    +- resolveTargets() --> política + disponibilidade + espaço livre
    |
    +- para cada nó alvo:
    |     +- ContainerClient.uploadReplica()
    |     |    +- nó SYSTEM --> escrita local directa (evita chamar-se a si)
    |     |    +- nó DEVICE --> HTTP POST /nodes/replicas/{id}
    |     |                     header X-Node-Token (autenticação do nó)
    |     +- FileReplica.status --> SYNCED | PENDING | FAILED
    |
    +- se nenhum nó aceitou --> NoStorageAvailableException + notificação
```

Notificação de falha: `REPLICA_FAILED` Notificação de falha: se a réplica falha num dispositivo, o u

3.5 Gestão honesta da disponibilidade (RF23–RF24)

Um PC pessoal desliga-se. O sistema assume isso em vez de fingir uptime:

Mecanismo	Implementação
Detecção de estado	`HeartbeatMonitorService` -> `ONLINE` / `OFFLINE` / `DEGRADED`
Comportamento offline	`OfflineBehavior` = `DENY` (negar) ou `CACHE` (cópia em cache)
Remoção segura	migrar réplicas antesantes de apagar (`migrateReplicas`, RF17.7)
Alertas de quota	notificação ao aproximar-se do limite (RF26)
Estado observável	ecrã de sincronização por ficheiro (RF28)

4. O diferencial

4.1 Comparativo

Capacidade	Drive / Dropbox / OneDrive	Este sistema

Ceder espaço do próprio PC como nó	[x]	[ok] (RF17)
Dois espaços geridos em conjunto	[x]	[ok] (RF18)
Escolha de destino no upload	[x]	[ok] (RF20)
Política de armazenamento configurável	[x]	[ok] (RF22)
Comportamento offline explícito	[x]	[ok] (RF24)
Migração de réplicas ao remover nó	[x]	[ok] (RF17.7)
Réplicas em múltiplos nós	parcial (opaco ao utilizador)	[ok] observável (RF28)
Versionamento	parcial	[ok] (RF13)
Links públicos com password	[ok]	[ok] (RF16)

4.2 Os três argumentos técnicos que sustentam o diferencial

A. A cifra é o que torna o modelo possível — não um extra

```
ficheiro --> AES-256-GCM --> container .dbs (opaco)
   |
   +- chave AES por ficheiro --> wrapKey(masterKey) --> FileMetadata.encryptedDataKey
```

- cada ficheiro tem a **sua própria chave AES**
- essa chave é **embrulhada** com a master key do servidor
- a master key vive só em `CLOUD_MASTER_KEY`, nunca no repositório

Consequência prática: os ficheiros sem a master key são inúteis — quem cede o disco vê blocos cifrados.

B. Indisponibilidade tratada como requisito, não como falha

Ao desligar, os projectos de "**cenário esperado**" assumem migração prévia; o caso **não esperado** exige réplicas já existentes, com política explícita de desvincular.

C. O nó é uma entidade de primeira classe, não um cliente burro

Cada nó tem identidade própria (quota, espaço usado, visibilidade) e **múltiplos dispositivos por conta** (PC + telemóvel), com detecção de agente, SO e tipo (`PERSONAL` / `SERVER`).

5. Decisões arquitecturais

5.1 Limitação conhecida: alcance de rede e NAT

- **Estado actual — limitação documentada.**

```
JAVA
private String nodeUrl(StorageNode node) {
    String sessionId = node.getSessionId();
    if (sessionId == null) return "http://localhost";
    return sessionId.startsWith("http") ? sessionId : "http://" + sessionId;
}
```

Isto produz o seguinte comportamento:

Cenário	Resultado
Backend e PC na mesma rede localmesma rede local	[OK] funciona
PC com IP públicoIP público ou túneltúnel (Tailscale, ngrok, Cloudflare Tunnel)	[OK] funciona
PC doméstico atrás de NATatrás de NAT, sem configuração	[X] falhafalha — o servidor não alcança `192.168.x.x`

Impacto: o sistema só funciona em redes onde o backend consegue iniciar ligações para os PCs dos utilizadores (tipicamente, redes com IP público ou configuração manual de NAT). Em ambientes domésticos ou empresariais típicos, a maioria dos agentes não pode ser registada nem receber comandos.

5.2 Solução proposta: ligação invertida (opção 2)

Em vez de o servidor **puxar** puxar do agente, o **agente puxa** a

```
HOJE (push do servidor)                PROPOSTO (pull do agente)

   Backend ------HTTP------> Agente        Agente ------HTTP------> Backend
        [x] bloqueado por NAT                   [ok] sempre permitido
                                               (saída 443)
```

Como funcionaria: Como funcionaria:

1. O agente arranca, abre uma **ligação de saída persistente** (WebSocket) para o backend
1. (long-polling) ou mantém um pedido HTTP aberto (**long-polling**). Quando o backend quer executar algo, envia o comando por essa ligação já existente
1. O agente recebe o comando, executa localmente e devolve o resultado pelo mesmo canal
1. O heartbeat passa a ser implícito: a ligação aberta **é** é o sinal de vida

Vantagens: Vantagens:

- funciona atrás de NAT, firewall e IP dinâmico — sem configuração do utilizador
- elimina a necessidade de túneis ou port forwarding
- o `sessionId` deixa de ser um endereço e passa a ser um identificador de canal
- o estado online/offline passa a ser fiável (ligação real, não ping)

Custos: Custos:

- o agente precisa de manter ligação aberta (reconnect com backoff)
- o backend precisa de um registo de canais/sessões activos
- comandos precisam de ser idempotentes e ter timeout

5.3 Alternativas consideradas

Opção	Mecanismo	Prós	Contras
1	Túnel automático no agente (Tailscale/Cloudflare)	mantém modelo actual	dependência externa; configuração no PC do utilizador
22	Ligação invertida (agente -> servidor)Ligação invertida (agente -> servidor)	zero configuração; funciona em qualquer redezero configuração; funciona em qualquer rede	mais trabalho no backend (sessões, filas)mais trabalho no backend (sessões, filas)

3	Sincronização em lote quando online	simples; tolera offline	réplicas não são imediatas; consistência eventual

Recomendada para o projecto. É a única que re

6. Modelo de segurança

Camada	Mecanismo
Autenticação de utilizador	JWT (HMAC256, `JWT_SECRET` >= 32 chars)
Autenticação do nó	`deviceToken` cifrado reversivelmente; header `X-Node-Token`
Pareamento de dispositivo	código de 15 min, uso único (`PairingCodeService`)
Cifra de ficheiros	AES-256-GCM por ficheiro, IV único
Protecção de chaves	chave por ficheiro embrulhada com `CLOUD_MASTER_KEY`
Autorização	roles (`ADMIN`), quotas, visibilidade do nó, `NodeAccess`
Transporte	HTTPS em produção

Regra de ouro (documentada no README): Regra de ouro (documentada `CLOUD_MASTER_KEY` não pode mudar sem migração, senão os ficheiros já armazenados ficam ilegíveis. A rotação de segredos é uma opção futura.

7. Riscos e trabalhos futuros

#	Item	Tipo
1	Ligação invertida (§5.2)	bloqueante para uso real em NAT
2	Documento de arquitectura de dados da replicação	os requisitos pedem-no explicitamente
3	Testes automáticos de frontend	ausentes (assumido no README)
4	Rotação segura da master key	hoje é destrutivo por desenho
5	Auditoria de acessos	RF29/RF30 a consolidar

8. Referências

- `docs/requisitos_sistema_armazenamento_nuvem.pdf`
- `docs/historias_usuario.pdf` — actores, épicos, histórias
- `README.md` — arranque, variáveis de ambiente, segurança
- `docs/OTP_EMAIL_SETUP.md` — configuração de OTP por email
- Código: `service/ContainerService`, `service/ReplicationService`, `entity/StorageNode`, `service/ContainerClient`, `int`

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem

Arquitetura do Sistema de Armazenamento em Nuvem